Programming Language Project

by

Aye Khin Khin Hpone (Yolanda Lim), st125970

Applegate Tun Oo, st126690

Win Htut Naing, st126687

AT70.07 — Programming Languages and Compilers

Assignment 2

Instructor: Prof. Phan Minh Dung

Teaching Assistant: Akraradet Sinsamersuk

Asian Institute of Technology

School of Engineering and Technology

Thailand

April 2026

ABSTRACT

This report presents the design and implementation of a statically-typed programming language developed for a programming languages and compilers assignment. The language supports four primitive types — Integer, Float, Boolean, and String — with type inference, arithmetic and boolean expressions, assignment statements, if-then-else and while-loop control flow, function abstraction with value parameter passing, and a built-in `print()` function. The implementation follows a classical compiler pipeline consisting of lexical analysis, recursive-descent parsing, abstract syntax tree construction, static type checking, and tree-walking interpretation. A command-line runner and a desktop user interface are provided so that script files can be loaded, executed, and inspected. The report describes the language grammar, the type inference algorithm, the implementation structure, and the testing used to validate the system.

CONTENTS

CHAPTER 1

INTRODUCTION

1.1 Scope of this report

This report outlines the development process of the programming language implemented in accordance with the specification of AT70.07 Programming Languages and Compilers — Assignment 2. The project was divided into three main components: the lexer and symbol table, the parser, and the type system with an interpreter. These were structured to be developed consecutively rather than simultaneously. By organising the work in sequence, each team member could focus on an assigned component independently before passing it to the next stage, which supported steady progress while reducing the need for overlapping work against other academic commitments.

1.2 Background and Motivation

Programming languages provide the formal structure through which algorithms are expressed and executed. A compiler or language processor is responsible for turning source code into a form that can be analysed or executed while also detecting errors as early as possible. Core compiler concepts such as lexical analysis, parsing, abstract syntax trees, symbol tables, static type checking, and interpretation are therefore fundamental topics in programming language education.

The motivation for this project is to apply those concepts in a compact but complete language implementation. By building a small statically-typed language, the project demonstrates how syntax, semantics, and execution can be connected in a coherent system. The assignment also provides an opportunity to study how type inference can improve usability while still preserving compile-time error checking.

1.3 Language Overview

The implemented language supports four primitive types: Integer, Float, Boolean, and String. It uses static typing with inference, so programmers do not need to write explicit type declara-

tions. Expressions include arithmetic operations with standard precedence and Boolean comparisons through equality and inequality. The statement-level constructs include assignment, if-then-else, while-loop, function definition, function call, return, and the built-in `print()` function.

Programs are processed through a lexer, a recursive-descent parser, a type checker, and an interpreter. The final system can be executed from the command line or through a desktop UI that allows the user to enter a script, run it, and inspect the resulting tokens, AST, inferred types, and output.

Chapter 2 presents the language design, token specification, grammar, and operator precedence rules. Chapters 3 and 4 cover the front-end implementation in pipeline order: lexical analysis and the symbol table, then recursive-descent parsing and the AST. Chapter 5 explains the type inference algorithm and the semantic rules used by the checker. Chapter 6 presents system architecture and implementation, including the end-to-end pipeline, project layout, and technology stack. Chapter 7 discusses testing and verification, and Chapter 8 concludes the report.

CHAPTER 2

LANGUAGE DESIGN AND GRAMMAR

2.1 Types

The language supports four primitive types. These types are sufficient to cover the assignment requirements while keeping the semantics simple and predictable.

2.1.1 Integer

The Integer type represents whole numbers such as `0`, `7`, and `42`. Integer literals are written as one or more decimal digits without quotation marks or a decimal point.

2.1.2 Float

The Float type represents real-valued numbers written with a decimal point, such as `3.14` or `20.5`. Float values are used in arithmetic expressions and may also arise through numeric promotion.

2.1.3 Boolean

The Boolean type represents logical truth values. The two Boolean literals are `true` and `false`. Boolean values are used in conditions and as results of comparison expressions.

2.1.4 String

The String type represents sequences of characters enclosed in double quotes, for example `"hello"`. Strings can be assigned to variables and printed through the built-in `print()` function. The surrounding quotes are stripped by the parser when the `Literal` node is created, so the value stored internally and printed at runtime is the bare character sequence without quotes.

2.1.5 Void

`Void` is an internal type used by the type system to represent the return type of functions that contain no `return` statement. It is not a type that programmers write explicitly; it exists solely inside the `LanguageType` enum so that the symbol table can record a well-defined type for

every function definition, including those that produce no value.

2.2 Static Typing

The language uses static typing, which means type errors are detected before execution rather than during program execution. No explicit variable type declarations are required. Instead, the type checker infers types from literals, assignments, operations, function calls, and return statements. Once a variable type is established, later assignments must remain consistent with that type.

2.3 Token Specification

Table 2.1 summarises the major token classes used by the language.

Table 2.1. Token Specification

Token Category	Example Lexemes
`INT_LITERAL`	`0`, `10`, `42`
`FLOAT_LITERAL`	`3.14`, `20.5`
`BOOL_LITERAL`	`true`, `false`
`STRING_LITERAL`	`"hello"`
`IDENTIFIER`	variable and function names such as `x`, `add`
Arithmetic operators	`+`, `-`, `*`, `/`
Comparison operators	`==`, `!=`
Assignment operator	`=`
Keywords	`if`, `else`, `while`, `def`, `return`, `print`
Delimiters	`( ) { } , ;`

2.4 Context-Free Grammar

The language grammar can be described by the context-free grammar $G = (V_N, V_T, P, S)$, where V_N is the set of non-terminals, V_T is the set of terminals, P is the set of production rules, and S is the start symbol `program`. The grammar is implemented through recursive-descent parsing functions.

2.4.1 Program Structure

```
program   -> statement* EOF
statement -> assignment ';'
       | if_stmt
       | while_stmt
       | func_def
       | print_stmt ';'
```

```
        | return_stmt ';'
        | expr ';'
```

2.4.2 *Arithmetic Expressions*

```
expr   -> term (('+' | '-') term)*
term   -> factor (('*' | '/') factor)*
factor -> INT_LITERAL
    | FLOAT_LITERAL
    | STRING_LITERAL
    | BOOL_LITERAL
    | IDENTIFIER
    | IDENTIFIER '(' args? ')'
    | '(' expr ')'
```

2.4.3 *Boolean Expressions*

```
bool_expr -> expr '==' expr
        | expr '!=' expr
        | BOOL_LITERAL
```

In the implementation, if `bool_expr` parses an expression and finds no following `==` or `!=` operator, the expression is returned as-is. The type checker then verifies that its inferred type is `Boolean`. This allows the grammar to remain simple while keeping the parser deterministic.

2.4.4 *Statements*

```
assignment  -> IDENTIFIER '=' expr
if_stmt     -> 'if' '(' bool_expr ')' block ('else' block)?
while_stmt  -> 'while' '(' bool_expr ')' block
print_stmt  -> 'print' '(' expr ')'
return_stmt -> 'return' expr
block       -> '{' statement* '}'
```

2.4.5 *Function Definitions and Calls*

```
func_def -> 'def' IDENTIFIER '(' params? ')' block
params   -> IDENTIFIER (',' IDENTIFIER)*
args     -> expr (',' expr)*
```

Function calls use value parameter passing. This means the argument values are evaluated before the call and copied into the function's local environment.

2.5 Operator Precedence and Associativity

Operator precedence is encoded structurally in the recursive-descent parser. Comparison is lowest, addition and subtraction are next, and multiplication and division have the highest precedence among binary operators. Arithmetic operators are left-associative.

Table 2.2. Operator Precedence (1 = lowest, 3 = highest) and Associativity

Precedence	Operators	Associativity
1 (lowest)	`==`, `!=`	non-associative (condition use only)
2	`+`, `-`	left-associative
3 (highest)	`*`, `/`	left-associative

CHAPTER 3

LEXICAL ANALYSIS AND SYMBOL TABLE

This chapter covers the responsibilities related to lexical and early semantic foundations of the language. It includes defining token structures and categories (such as keywords, operators, identifiers, and literals), implementing the lexer to transform source characters into tokens, and establishing the symbol table to manage variable and function storage along with their associated types. Together, these components ensure that raw source input is systematically organised into meaningful structures that can be used reliably by later stages of the compiler.

3.1 Token representation

The token system, written in `components/tokens.py`, establishes the vocabulary and structure of all lexical elements in the language. It specifies both the types of tokens that can exist and the structure each token must follow. This serves as a canonical contract between the lexer and the parser, ensuring consistency across all stages of the compiler.

Table 3.1 summarises the main categories, example lexemes, and typical `TokenType` values.

Table 3.1. Token categories and representative `TokenType` values

Category	Example lexemes	Representative `TokenType`
Literals	`42`, `3.14`, `true`, `"hello"`	`INT_LITERAL`, `FLOAT_LITERAL`, `BOOL_LITERAL`, `STRING_LITERAL`
Identifiers	`x`, `counter`, `add`	`IDENTIFIER`
Keywords	`if`, `else`, `while`, `def`, `return`, `print`	keyword-specific token variants
Operators	`+`, `-`, `*`, `/`, `=`, `==`, `!=`	`PLUS`, `MINUS`, `STAR`, `SLASH`, `ASSIGN`, `EQUAL_EQUAL`, `BANG_EQUAL`
Delimiters	Parentheses, braces, comma, and semicolon (see lexer delimiter tokens)	delimiter token variants
End marker	end of source input	`EOF`

Token types (from class `TokenType`) are defined using an `Enum` with `auto()` to automatically assign unique values. This avoids reliance on raw strings and ensures safer comparisons during parsing. The token categories include literals, identifiers, keywords, operators, delimiters, and a special end-of-file (`EOF`) token. The `EOF` token is appended after all input is processed,

allowing the parser to detect the end of input explicitly rather than encountering an unexpected termination.

Each token is represented as an immutable dataclass (`frozen=True`), meaning its fields cannot be modified after creation. Each token contains four key pieces of information: its kind (`token_type`), its lexeme (the exact substring from the source code), and its source position (`line`, `column`). This structured representation allows later stages to process the program in terms of syntax and errors without handling raw text directly. Immutability improves reliability by preventing accidental modification during later processing.

```
@dataclass(frozen=True)
class Token:
    token_type: TokenType
    lexeme: str
    line: int
    column: int
```

3.2 Lexer implementation

The lexer, implemented in `components/lexica.py`, is a hand-written scanner that converts raw source text into a sequence of tokens. Unlike generated lexers (for example SLY lexer mode, or Lex and Flex), this implementation uses explicit control flow (`if` and `while`) to process input.

The lexer maintains internal state including the source string, current position, start of the current token, line number, and column index. The column index is tracked per line and resets upon encountering a newline, while a separate `token_column` records the starting position of each token for accurate error reporting.

The main function, `tokenize`, iterates through the source code, repeatedly invoking `_scan_token` to identify the next token. Tokens are appended to a list, and an `EOF` token is added at the end. Whitespace is skipped, while invalid input results in a `LexerError`.

```
def tokenize(self) -> list[Token]:
    tokens: list[Token] = []
    while not self._is_at_end():
        self.start = self.current
        self.token_column = self.column
        token = self._scan_token()
        if token is not None:
            tokens.append(token)
    tokens.append(Token(TokenType.EOF, "", self.line, self.column))
    return tokens
```

Token recognition follows a structured approach. Single-character tokens are mapped using a lookup table. Multi-character operators such as `==` and `!=` are handled using lookahead via `_match`. String literals are processed by `_string`, which reads characters until a closing quotation mark or the end of input is reached, with error handling for unterminated strings. Numeric literals are handled by `_number`, which distinguishes integers and floating-point values based on the presence of a decimal point followed by digits. Identifiers are processed greedily, consuming alphanumeric characters and underscores, and then checked against the `KEYWORDS` map to determine whether they are reserved words or user-defined names. Notably, `true` and `false` are also entries in this map, mapping to `TokenType.BOOL_LITERAL` rather than a keyword-specific type. This means boolean literals are recognised through the same keyword-fallback mechanism as reserved words, preventing them from being scanned as user-defined identifiers.

```
KEYWORDS: dict[str, TokenType] = {
    "if": TokenType.IF,
    "else": TokenType.ELSE,
    "while": TokenType.WHILE,
    # ...
    "true": TokenType.BOOL_LITERAL,
    "false": TokenType.BOOL_LITERAL,
}
```

```
SINGLE_CHAR_TOKENS: dict[str, TokenType] = {
    "+": TokenType.PLUS,
    "-": TokenType.MINUS,
    "*": TokenType.STAR,
    "/": TokenType.SLASH,
    # ...
}
```

Helper functions such as `_advance`, `_peek`, and `_peek_next` manage character consumption and lookahead, while `_is_at_end` determines when input has been fully processed. Error messages include precise line and column information, aiding debugging and diagnostics.

Table 3.2 lists the main functions and data structures in `components/lexica.py` and their roles.

Table 3.2. Key functions and data in `lexica.py`

Function / data	Responsibility	Output / effect
`tokenize()`	Iterates through source, scans tokens, appends `EOF`	ordered token list
`_scan_token()`	Dispatches recognition based on character and lookahead	next token or skip
`_number()`	Parses numeric sequences, distinguishes int vs float	numeric literal token
`_string()`	Parses quoted text, checks termination	string token or error
`_identifier()`	Parses identifiers, resolves keyword vs user-defined	keyword or identifier token
`KEYWORDS` map	Maps reserved words to token types	token type selection
`SINGLE_CHAR_TOKENS` map	Maps single-character symbols	token type selection
`_advance()`, `_peek()`, `_match()`	Character consumption and lookahead	scanner state update

3.3 Symbol table and semantic structure

The symbol table, defined in `components/symbol_table.py`, operates at a higher level than the lexer. While the lexer processes raw text into tokens, the symbol table manages semantic information such as variable and function names, their types, and their scope.

The symbol table is implemented as a scoped structure, where each scope maintains its own dictionary of symbols and optionally references a parent scope. This enables proper handling of nested blocks and variable shadowing. Symbols are introduced through definition functions and retrieved through lookup operations, which search recursively through parent scopes if needed.

```
def lookup(self, name: str) -> Symbol:
    symbol = self._symbols.get(name)
    if symbol is not None:
        return symbol
    if self.parent is not None:
        return self.parent.lookup(name)
    raise SymbolTableError(f"Undefined symbol: {name}")
```

The type system is represented using a `LanguageType` enumeration, which includes fundamental types such as Integer, Float, Boolean, String, and Void. Symbols are modelled using an inheritance hierarchy: the base `Symbol` class stores a name and type, while subclasses such as `VariableSymbol` and `FunctionSymbol` include additional attributes. Variables track initialization state, while functions store parameter lists and return types. Table 3.3 summarises the main building blocks.

Table 3.3. Symbol table components and roles

Component	Stored information	Role in semantic analysis
Scoped symbol table	Symbols in current and parent scope	Supports nesting and shadowing via recursive lookup
`LanguageType` enum	Integer, Float, Boolean, String, Void	Defines the type system
`Symbol` (base)	Name and declared or inferred type	Base abstraction for entries
`VariableSymbol`	Variable metadata (including initialization state)	Tracks correct usage before assignment
`FunctionSymbol`	Parameters and return type	Validates calls and return consistency
`lookup(name)`	Recursive scope search	Resolves identifiers or reports undefined
`SymbolTableError`	Semantic error details	Reports duplicates, undefined symbols, invalid operations

Errors related to semantic violations are handled via `SymbolTableError`. These include duplicate declarations within the same scope, references to undefined symbols, and invalid operations such as attempting to mark a non-variable symbol as initialized.

3.4 Role of the symbol table in the pipeline

The symbol table is constructed during the type-checking phase rather than during lexical analysis. This guideline is followed in the implementation. The type checker initialises a global symbol table and populates it while traversing the AST, including defining variables and functions, managing nested scopes, and validating type correctness.

```
def run_pipeline(source_code: str) -> PipelineResult:
    tokens = Lexer(source_code).tokenize()
    tree = Parser(tokens).parse()
    ast_text = ASTPrinter().print(tree)
    checked_scope = TypeChecker().check(tree)
    outputs: list[str] = []
    Interpreter(output_callback=outputs.append).execute(tree)
    return PipelineResult(
        tokens=tokens,
        ast_text=ast_text,
        checked_scope=checked_scope,
        outputs=outputs,
    )
```

Once type checking is complete, the populated symbol table is returned as part of the pipeline result (`checked_scope`). The checker creates child scopes for blocks and functions during analysis, but the displayed `checked_scope` table represents the global scope entries returned

by the pipeline. It can then be displayed or used for further analysis.

This separation of concerns keeps lexical analysis focused on tokenization, while semantic meaning is enforced during type checking. For user-defined functions, definitions are first registered in the global scope with placeholder parameter and return types. These are inferred from the first valid call and refined through body checking; subsequent calls must match the inferred parameter types.

CHAPTER 4

RECURSIVE-DESCENT PARSING AND THE ABSTRACT SYNTAX TREE

This chapter presents the implementation of the recursive-descent parser and the associated abstract syntax tree (AST) model. Starting from lexer-produced tokens, the parser constructs a hierarchical `Program` AST that represents expressions, statements, and control-flow structure in a form suitable for downstream semantic analysis and execution. Implemented syntax coverage includes arithmetic expressions, assignments, if-then-else, while, function definitions, function calls, return, and built-in `print(...)` statements. Equality operators (`==`, `!=`) are parsed in condition contexts (if/while) through the parser's boolean-expression path.

4.1 Abstract syntax tree representation

The AST is defined as a collection of Python dataclasses representing the structural elements of the programming language. It serves as an intermediate representation between parsing and later stages such as type checking and interpretation.

A key distinction exists between tokens and AST nodes. Tokens are flat and describe individual lexical elements such as operators or identifiers. In contrast, the AST is hierarchical. For example, while a token may represent the symbol `+`, a `BinaryOp` node represents an entire addition operation, including its left and right operands as subtrees.

All AST nodes inherit from a common base class, allowing them to store positional metadata (`line`, `column`). This ensures that errors detected in later stages can still reference precise locations in the original source code.

```
@dataclass
class ASTNode:
    """Base class for every AST node. Carries source location for
       diagnostics."""
    line: int = field(default=0, kw_only=True)
    column: int = field(default=0, kw_only=True)
```

4.1.1 *Expressions*

Expressions represent computations or values within the program. A `Literal` node stores values known at parse time, including integers, floating-point numbers, booleans, and strings. String literals are stored without surrounding quotation marks, as these are removed during parsing.

```
if tok.token_type == TokenType.STRING_LITERAL:
    self._advance()
    return Literal(value=tok.lexeme[1:-1], line=tok.line, column=tok.
        column)
```

An `Identifier` node represents the use of a variable name within an expression, indicating evaluation rather than declaration. A `BinaryOp` node models arithmetic and comparison operations such as addition, subtraction, multiplication, division, and equality checks, storing the operator along with references to left and right operands. A `FunctionCall` node represents function invocation, consisting of the function name and a list of argument expressions.

```
@dataclass
class BinaryOp(ASTNode):
    """An arithmetic or comparison binary operation.

    op is one of: '+' '-' '*' '/' '==' '!='
    Example:  a + b    x == 0
    """
    op: str
    left: ASTNode
    right: ASTNode
```

4.1.2 *Statements*

Statements represent actions or control flow within the program. An `Assign` node models variable assignment, pairing a variable name with a value expression, while `Print` and `Return` nodes each store a single expression and represent output and function return operations respectively. A `Block` node groups multiple statements within braces, representing a scoped execution sequence.

Control flow is handled via `If` and `While` nodes. The `If` node stores a condition, a then-block, and an optional else-block, while `While` stores a loop condition and body block. Function definitions are represented by `FunctionDef` nodes, which include the function name, a list of parameter names, and a body block. Parameter types are intentionally not assigned at parse time, as type inference is handled later by the type checker. Finally, `Program` serves as the

AST root, containing all top-level statements.

4.2 Parser implementation

The parser consumes the list of tokens generated by the lexer and produces a single `Program` AST node. It is implemented as a hand-written recursive-descent parser based on the project grammar. This avoids parser generators and instead relies on explicit functions for each grammar rule.

The parser maintains internal state consisting of the token list and a position index (`self._pos`) indicating the current token. Tokens are accessed via helper methods such as `_peek` and `_advance`. Parsing errors are reported using a custom `ParseError` exception with line and column information consistent with lexer diagnostics.

```
def parse(self) -> Program:
    """Parse the full token stream and return the root Program node."""
    line, col = self._peek().line, self._peek().column
    statements: list[ASTNode] = []
    while not self._is_at_end():
        statements.append(self._statement())
    return Program(statements=statements, line=line, column=col)
```

In addition to the main `parse()` entry point, the parser's statement dispatcher (`_statement`) implements a deterministic decision order over token patterns: function definition, conditional, loop, return, print, assignment, then expression statement fallback. This design is important because it prevents ambiguity between syntactically similar forms. In particular, assignments are recognized using one-token lookahead (`IDENTIFIER` followed by `=`), while identifier-led expressions that are not assignments are parsed as expression statements and must still end with a semicolon. This gives the parser full coverage of both declarative-style and expression-driven top-level statements without requiring a parser generator.

4.2.1 Statement parsing

The main parsing loop repeatedly processes statements until `EOF`, with the statement type determined via dispatch on the current token. Function definitions, conditionals, loops, returns, and prints each have dedicated parsing methods. Assignments are detected using one-token lookahead (`IDENTIFIER` followed by `ASSIGN`), preventing misclassification of function calls as assignments. If no specific statement pattern matches, the parser falls back to an expression statement and enforces a terminating semicolon.

```
# assignment: IDENTIFIER "=" expr ";"
if (
    tok.token_type == TokenType.IDENTIFIER
    and self._peek_next().token_type == TokenType.ASSIGN
):
    return self._assignment()
# fallback: bare expression statement  expr ";"
node = self._expr()
self._expect(TokenType.SEMICOLON, "Expected ';' after expression")
```

4.2.2 *Expression parsing and precedence*

Expressions are parsed in layered precedence levels: `_expr` handles `+` and `-`, `_term` handles `*` and `/`, and `_factor` handles literals, identifiers and function calls, and parenthesized expressions. Boolean conditions for `if` and `while` are parsed by `_bool_expr`, which recognises `==` and `!=` after parsing the left-hand arithmetic expression. As a result, equality parsing is condition-focused in the current grammar implementation rather than a general infix operator available in every expression position. Function calls are recognised when an identifier is followed by `(`, with argument lists parsed as comma-separated expressions. Parenthesized expressions are supported to override precedence.

```
def _expr(self) -> ASTNode:  # +,-
    ...
def _term(self) -> ASTNode:  # *,/
    ...
def _factor(self) -> ASTNode:  # literals, identifiers/calls, grouped
    expr
    ...
```

Figure 4.1 shows the AST produced for the statement `z = x + y * 2;`. Because `_term` handles `*` before `_expr` handles `+`, the multiplication sub-tree is nested one level deeper, encoding the higher precedence of `*` over `+` without any explicit priority table at runtime.

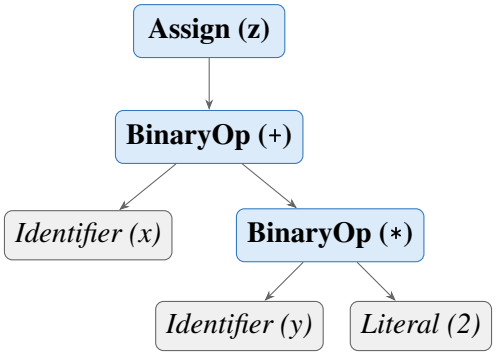

Figure 4.1. AST for `z = x + y * 2;`. Multiplication is a deeper sub-tree than addition, encoding higher precedence through tree structure rather than a runtime priority table.

Expression parsing is intentionally split into layered methods to enforce precedence and associativity in a transparent way. Parenthesized expressions are parsed explicitly and re-enter `_expr`, allowing precedence override where required by source syntax. Boolean conditions for `if` and `while` are parsed through `_bool_expr`, which accepts equality and inequality comparisons but can also return a non-comparison expression node; semantic enforcement that a condition is truly Boolean is deferred to the type checker.

4.2.3 Helper functions and lookahead

As described above, the parser uses `_check`, `_expect`, `_peek`, `_peek_next`, `_advance`, and `_is_at_end` for token navigation and validation. Lookahead is central for ambiguity resolution: assignment versus function call is resolved by inspecting the next token; comparison operators are recognised after parsing the left expression in `_bool_expr`. Parser errors follow a consistent diagnostic format: `[line X, col Y] ParseError: <message>`.

The parser also includes dedicated list parsers for formal parameters (`_params`) and call arguments (`_args`), both implemented as comma-separated sequences with optional emptiness. This ensures consistent handling of signatures such as `def f()` and `def f(a, b)` as well as invocations like `f()` and `f(x, y+1)`.

4.3 AST printing

For debugging and reporting, an AST printer converts the tree into a readable indented string, integrated into the pipeline output. The printer uses visitor-style dynamic dispatch, mapping node class names to methods such as `_visit_If` and `_visit_BinaryOp`, with output built as multi-line text where indentation reflects tree depth. Node formatting emphasises structure:

`Program` and `Block` include statement counts, `BinaryOp` explicitly shows left and right subtrees, and `FunctionCall` prints each argument by index. `Literal` includes both value and runtime Python type names for clarity.

4.4 Parser and type checker boundary

A deliberate architectural boundary is maintained between syntax construction and semantic validation. The parser is responsible for producing a structurally valid AST with accurate source locations, while type correctness and semantic constraints are checked later by the type checker. For example, `_bool_expr` may syntactically accept a non-Boolean expression as a condition node, but rejection of non-Boolean control-flow conditions is handled during semantic analysis. This separation keeps parser logic focused on grammar conformance and allows a clear responsibility split across team member contributions.

CHAPTER 5

TYPE INFERENCE ALGORITHM

5.1 Overview

Type inference in this project means that the programmer does not write explicit type declarations for variables or functions. Instead, the compiler infers types from assignments, literals, operations, function calls, and return statements. This keeps the language simple while still preserving static type safety.

The type checker runs after parsing. It receives the AST and validates the program before execution. Any inconsistent use of types is reported as a type error, and execution is stopped.

The interpreter stage does not repeat this validation: it walks the same AST with ordinary Python representations of values. Static type information remains in the symbol table produced by the checker; run-time values are not separately labelled with `LanguageType`, except that `print()` formats output by inspecting the value's Python type.

5.2 Type Environment

The type environment is represented by the symbol table. Conceptually, it is a mapping

$$\Gamma : \textit{Identifier} \rightarrow \textit{Type}$$

The environment grows as the checker visits the AST. On first assignment, a variable name is inserted with its inferred type. When a new block or a function call is entered, a child environment is created so that nested scopes can be checked independently while still accessing outer bindings when needed.

5.3 Inference Rules

5.3.1 Literals

Literal nodes have direct types:

$$\Gamma \vdash 42 : \texttt{Integer}$$

$$\Gamma \vdash 3.14 : \texttt{Float}$$

$$\Gamma \vdash \texttt{true} : \texttt{Boolean}$$

$$\Gamma \vdash \text{“}\texttt{hi}\text{”} : \texttt{String}$$

5.3.2 Arithmetic Expressions

The arithmetic operators `+`, `-`, `*`, and `/` accept only numeric operands. If both operands are `Integer`, the result is `Integer`, except for division, which produces `Float`. If at least one operand is `Float`, the result is `Float`. Any non-numeric operand causes a type error.

5.3.3 Boolean Expressions

The comparison operators `==` and `!=` always produce `Boolean`. Numeric comparison is allowed for arithmetic expressions. The current implementation also permits Boolean-to-Boolean comparison because the sample programs use `flag!=false`. Comparisons involving `String` operands are rejected (so string equality is not part of the type system). Other combinations are rejected.

5.3.4 Assignment Statement

Assignments define variable types on first use. If a variable does not yet exist in the type environment, the checker infers the type of the right-hand side and inserts the variable into the environment. If the variable already exists, the new value must have the same type.

$$\text{If } x \notin \Gamma \text{ and } \Gamma \vdash e : T, \text{ then } \Gamma, x : T \vdash x = e$$

$$\text{If } x : T \in \Gamma \text{ and } \Gamma \vdash e : T, \text{ then } \Gamma \vdash x = e$$

5.3.5 If-Then-Else

The condition of an `if` statement must be `Boolean`. The then-block and else-block are checked in child scopes so that local operations can be validated cleanly.

5.3.6 *While-Loop*

The condition of a `while` loop must also be `Boolean`. The loop body is checked in a child scope and must not violate the types of variables already introduced in the surrounding environment.

5.3.7 *Function Definition and Call*

Function parameter types are inferred from the first call to the function. After that first call, subsequent calls must use arguments of the same types. The return type is inferred from the `return` statements in the function body. If the function returns inconsistent types, the checker reports an error.

Recursive calls are intentionally rejected in this implementation because they were not required by the assignment and would require additional inference and control-flow analysis.

5.4 Type Error Reporting

Type errors are reported with source positions so that the user can locate the problem quickly. The format used by the system is:

```
[line X, col Y] TypeError: message
```

Examples include assigning a `String` to an `Integer` variable, using a non-Boolean condition in an `if` statement, or supplying the wrong argument type to a function.

5.5 Example Type Inference Traces

Consider the following program fragment:

```
x = 10;
y = 20.5;
z = x + y;
print(z);
```

The checker infers:

- `x: Integer`
- `y: Float`
- `x + y: Float`

- `z: Float`

For a function example:

```
def add(a, b) {
        return a + b;
}

result = add(2, 3.5);
```

The first function call infers `a: Integer` and `b: Float`. The expression `a + b` is therefore `Float`, so the function return type is inferred as `Float`. The variable `result` is also inferred as `Float`.

CHAPTER 6

SYSTEM ARCHITECTURE AND IMPLEMENTATION

This chapter describes how the implementation is organised: the end-to-end processing flow, repository layout, technology choices, and the main components that realise static analysis and execution (type checker, interpreter, and user interfaces). Detailed treatment of lexical analysis, the symbol table, and parsing appears in Chapters 3 and 4; Chapter 5 presents the type-inference rules that the type checker implements.

6.1 Compiler pipeline

The system follows a four-stage pipeline. Lexical and syntactic processing are kept separate from semantic analysis and execution, which simplifies testing and localises errors to the appropriate stage.

The overall flow of data through the system is shown in Figure 6.1.

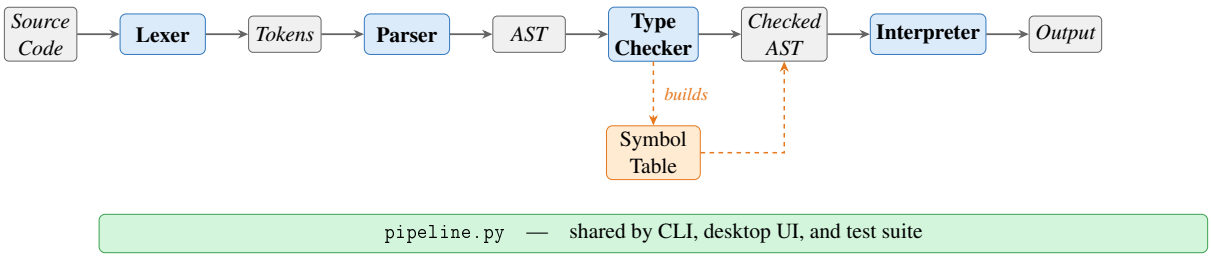

Figure 6.1. Compiler pipeline overview.

Source characters are scanned by the lexer into a token stream. The parser consumes tokens and builds an AST that reflects program structure (expressions, statements, and control flow). The type checker then walks the AST, populates and uses the symbol table, infers types, and validates scope and type rules. Only programs that pass the type checker proceed to interpretation. The interpreter evaluates the checked tree using ordinary Python values; it does not re-run the type rules, since correctness at runtime is guaranteed by the prior static pass. All four stages are orchestrated through `pipeline.py`, which is shared by the command-line runner, the desktop UI, and the test suite.

6.1.1 Entry points and shared pipeline

All stages are orchestrated by `components/pipeline.py`, which is shared by the command-line runner, the desktop UI, and the test suite. From the project root, `python3 main.py` runs a script (or built-in sample) and prints the stages in order: tokens, AST text, type table, and execution output. `python3 ui.py` runs the Tkinter workbench: the same pipeline is used internally, with each stage shown in a separate tab so the user can inspect results without altering the underlying flow.

6.2 Project structure

The main source files and their responsibilities are summarised in Table 6.1.

Table 6.1. Main source files

File	Responsibility
`main.py`	Command-line runner for sample code or a script file
`ui.py`	Desktop UI for editing and running a script
`components/tokens.py`	Token classes and token categories
`components/lexica.py`	Lexical analysis
`components/ast_nodes.py`	AST node definitions
`components/parser.py`	Recursive-descent parser
`components/ast_printer.py`	AST renderer for debugging and reporting
`components/symbol_table.py`	Scoped symbol table and semantic types
`components/type_checker.py`	Static type checker and inference
`components/interpreter.py`	Tree-walking interpreter
`components/pipeline.py`	Shared pipeline for CLI, UI, and tests

6.3 Technology stack

The project uses Python and standard-library tools only (Table 6.2).

Table 6.2. Technology stack

Tool	Purpose
Python 3	Core implementation language
`dataclasses`	AST and runtime helper structures
`tkinter`	Desktop graphical interface
`unittest`	Automated testing

6.4 Type checker

The `TypeChecker` class implements the inference and checking rules described in Chapter 5. It traverses the AST before execution, uses the symbol table to record variable and function information, infers types on first assignment, validates arithmetic and comparison expressions, enforces Boolean conditions for control flow, infers function parameter types from the first call, and infers function return types from `return` statements. Programs that fail these rules are rejected before interpretation.

6.4.1 Scope of the implemented type system

In scope terms, this component turns a syntactically valid AST into a semantically valid program:

- static typing rules for arithmetic, comparison, assignment, control flow, and functions,
- type checking for expressions and statements,
- variable type inference from first assignment,
- function parameter inference from the first call,
- function return type inference from `return` statements,
- source-positioned type errors when a program violates a typing rule.

6.5 Interpreter

The interpreter is a tree-walking evaluator over the checked AST. It maintains nested runtime environments for variables and function calls. Assignments write through the current or nearest enclosing scope; `if` selects a branch; `while` repeats until its condition becomes false; function calls bind arguments by value and return through an internal return mechanism.

The built-in `print()` function outputs both value and runtime type, such as `3 : Integer` or `"plc" : String`.

6.5.1 Scope of the implemented runtime

The runtime stage provides observable behaviour and integrates with the shared pipeline:

- assignment execution,
- `if` execution,
- `while` execution,
- function execution with value parameter passing,
- `print()` execution with immediate output,
- execution behind the same `run_pipeline` path used by the CLI, UI, and tests.

6.6 Graphical user interface

The workbench is implemented with `tkinter`. The layout required for the assignment is preserved: the script is edited on the left, and pipeline outputs appear on the right. The right-hand side surfaces the same stages as the command-line printout, but in a tabbed panel so the user can switch between views without rerunning the pipeline for each kind of inspection.

CHAPTER 7

TESTING AND VERIFICATION

7.1 Test Cases

Testing was performed through both manual execution and automated regression tests. Manual runs were used to inspect the full pipeline from source code to output, while automated tests were added for the semantic and runtime behaviour of the implemented type checker and interpreter.

Table 7.1. Representative Test Cases

Category	Program Behaviour	Expected Result
Arithmetic	Evaluate mixed numeric expressions	Correct value and inferred type
If-then-else	Choose one branch from a Boolean condition	Only matching branch runs
While-loop	Print during repeated execution	One output line per iteration
Functions	Infer parameters and return type	Valid call and correct return value
Type mismatch	Reassign incompatible type	Type error before runtime

7.1.1 Lexer and Symbol Table

The lexer was verified manually by running the full pipeline and inspecting Stage 1 token output for programs that exercise every token category: integer and float literals, Boolean literals, string literals, all six keywords, two-character comparison operators (`==`, `!=`), single-character operators, and delimiters. Identifier scanning with keyword fallback was confirmed by checking that `true`, `false`, and reserved words are emitted with their keyword token types while non-keyword names produce `IDENTIFIER` tokens.

Error paths were also checked: supplying a bare `!` character raises a `LexerError` with a source position and a message suggesting `!=`, and an unterminated string literal raises a `LexerError` citing the opening line.

The symbol table was verified through the type checker and the Stage 3 output. Duplicate

variable definitions were confirmed to raise a `SymbolTableError` before execution. Scope isolation was verified with nested blocks: a variable defined inside an `if` or `while` body is not visible in the outer scope after the block ends. The `format_table()` output was checked against the expected column layout (name, kind, type, initialisation status, parameters) for both variable and function symbols.

7.1.2 Parser and AST

The parser was verified manually by running the full pipeline and inspecting Stage 2 AST output. The following cases were checked:

- arithmetic expressions with mixed precedence (e.g. `x + y * 2`) produce a tree where multiplication is a deeper sub-tree than addition, confirming the `_expr`/`_term`/`_factor` precedence encoding,
- assignment versus expression statement dispatch: `x = 5;` produces an `Assign` node while `add(1,2);` produces a `FunctionCall` node at the statement level, confirming the IDENTIFIER+ASSIGN lookahead,
- `if` with and without an `else` clause: the resulting `If` node has a non-null `else_block` only when the clause is present,
- function definitions store only parameter names in the `FunctionDef` node; the AST printer output was inspected to confirm no type annotations are present at this stage,
- string literals in `Literal` nodes have their surrounding double-quotes stripped; `print("hello")` prints `hello : String` at runtime,
- parse errors: missing semicolons, unmatched parentheses, and unexpected tokens all produce `[line X, col Y] ParseError:` messages with correct source positions.

7.1.3 Arithmetic Expressions

Arithmetic expressions were tested using both integer-only and mixed integer-float programs. These tests verified precedence and numeric promotion. Expressions such as `x + y * 2` confirmed that multiplication is applied before addition.

7.1.4 Boolean Expressions

Boolean expressions were tested inside `if` and `while` conditions. The checker verified valid comparison operands, and the interpreter used the resulting Boolean values for control flow.

7.1.5 Assignment Statements

Assignments were tested for both first-use inference and reassignment checks. For example, a variable assigned `5` and later assigned `"hello"` correctly triggers a type error.

7.1.6 If-Then-Else

Conditional programs were used to verify that exactly one branch is executed and that non-Boolean conditions are rejected before runtime.

7.1.7 While-Loop

While-loops were tested with a countdown example. This confirmed that the loop body executes repeatedly and that `print()` emits output progressively on each iteration rather than only once at the end.

7.1.8 Functions

Function tests covered parameter passing, local execution, and return values. The implementation infers parameter types from the first call and then enforces that signature for later calls.

7.1.9 Print

The built-in `print()` function was tested with all supported runtime types. The output includes the printed value together with its type, such as `29.5 : Float` and `"plc" : String`.

7.2 Type Error Detection

The checker was verified with invalid programs involving incompatible assignment, invalid arithmetic operands, wrong function arguments, and non-Boolean conditions. In each case the system stopped before execution and reported a source-positioned type error.

7.3 Automated Test Suite

Automated regression tests were added in `tests/test_member3_pipeline.py` using the `unittest` framework. The suite currently covers:

- progressive output during while-loop execution,
- assignment type mismatch detection,
- function parameter and return type inference.

The tests can be run using:

```
python3 -m unittest discover -s tests -v
```

7.4 Error Handling

The language system reports errors at three main stages:

- lexical errors for malformed characters or unterminated strings,
- parse errors for invalid syntax such as missing semicolons,
- type errors for programs that violate static typing rules.

Runtime errors are also reported with source positions whenever execution reaches an invalid state such as an undefined variable reference.

CHAPTER 8

CONCLUSION

This project delivered a working statically-typed programming language with the required primitive types: Integer, Float, Boolean, and String. The language supports arithmetic expressions, equality and inequality checks, assignments, if-then-else statements, while-loops, function abstraction with value parameter passing, return statements, and the built-in `print()` function.

The final system follows a clean compiler-style architecture. The lexer creates tokens, the parser builds the AST, the type checker performs static inference and validation, and the interpreter executes the program. This staged design made it easier to isolate syntax errors, type errors, and runtime behaviour.

One of the main outcomes of the project is the successful implementation of static typing without explicit declarations. Variable types are inferred from assignments, and function signatures are inferred from calls and return statements. Invalid programs are rejected before runtime. In addition, the system includes both a command-line runner and a desktop UI so that scripts can be entered, loaded, executed, and inspected conveniently.

The automated tests added for the semantic and runtime stage pass and confirm correct behaviour for loop execution, assignment mismatch detection, and function type inference. These checks provide a baseline for future extensions.

Future work could include recursive functions, richer data structures, additional operators, and code generation to a lower-level target. However, the current implementation already satisfies the main functional requirements of the assignment.

REFERENCES

Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, MA, 2nd edition, 2006.

Robert Nystrom. *Crafting Interpreters*. Genever Benning, Seattle, WA, 2021.

Python Software Foundation. Python documentation: dataclasses — data classes. `https://docs.python.org/3/library/dataclasses.html`, 2026a. Accessed: April 2026.

Python Software Foundation. Python documentation: Tkinter — python interface to tcl/tk. `https://docs.python.org/3/library/tkinter.html`, 2026b. Accessed: April 2026.

Python Software Foundation. Python documentation: unittest — unit testing framework. `https://docs.python.org/3/library/unittest.html`, 2026c. Accessed: April 2026.

APPENDIX A: SOURCE CODE AVAILABILITY

The complete source code for this project is available on GitHub:

https://github.com/The-Token-Trio/125970-126690-126687-Assignment2-PLC

The repository contains the lexer, parser, AST nodes, symbol table, type checker, interpreter, shared execution pipeline, user interface, tests, and the full report source. The project can be run either from the command line through `main.py` or from the desktop UI through `ui.py`.

The main execution commands are:

```
python3 main.py
python3 ui.py
python3 -m unittest discover -s tests -v
```

No external Python dependencies are required for the implementation. The Python side of the project uses only the standard library.

APPENDIX B: TEAM MEMBERS AND CONTRIBUTIONS

Name	Student ID	Programme
Aye Khin Khin Hpone (Yolanda Lim)	st125970	Master's
Applegate Tun Oo	st126690	Master's
Win Htut Naing	st126687	Master's

Name	Student ID	Assignment Scope
Aye Khin Khin Hpone (Yolanda Lim)	st125970	Static typing rules; type checking; assignment execution; if execution; while execution; function execution; `print()` execution
Applegate Tun Oo	st126690	Token definitions; keywords and operators; identifiers and literals; variable/function storage; type storage
Win Htut Naing	st126687	Arithmetic expressions; boolean expressions; assignment statements; if-then-else; while-loop; function definitions; function calls; `print()` syntax